

BlockPIM: Optimizing Memory Management for PIM-enabled Long-Context LLM Inference

Zhichun Li^{1,2}, Jun Zhou^{1,2}, Xueqi Li^{1,*}, Ninghui Sun^{1,2}

¹State Key Lab of Processors, Institute of Computing Technology, CAS ²University of Chinese Academy of Sciences
 {lizhichun23z, zhoujun23s, lixueqi}@ict.ac.cn, snh@ncic.ac.cn

Abstract—Processing-In-Memory (PIM) architectures alleviate the memory bottleneck in the decode phase of large language model (LLM) inference by performing operations like GEMV and Softmax in memory. However, the fragmented data layout in current PIM architectures limits end-to-end acceleration for long-context LLMs. In this paper, we propose BlockPIM, a cross-channel block memory layout strategy that maximizes memory utilization and eliminates the context length constraint. Additionally, we introduce a cross-channel attention computation scheme that is compatible with the current architecture to support distributed attention operations on BlockPIM. Experimental results demonstrate that our approach achieves a 62% average throughput increase compared to existing state-of-the-art PIM solutions, enabling efficient and scalable deployment of large language models on PIM architectures.

I. INTRODUCTION

The large language models (LLMs) inference process consists of two stages: the *prefill stage* and the *decode stage*. The prefill stage processes the input prompt to generate initial context, while the decode stage follows, where the model generates tokens sequentially using the learned context and previous tokens to predict the next one. As demonstrated before, the main bottleneck is the decode stage and the performance is heavily constrained by memory bandwidth due to low compute density. While batching techniques can improve compute density for some operations by reusing parameters, the attention operation still suffers from extensive memory access between the processing units and the memory, making the memory a major performance bottleneck.

PIM (Processing-In-Memory) architectures is considered an ideal technological path [1]–[4], which integrates specialized processing elements (PEs) near memory banks or on buffer dies, enabling memory-intensive operators such as GEMV and Softmax—key components of attention—to be offloaded. However, to improve parallelism, fully utilize internal bandwidth (e.g., 242 TB/s in AttAcc [3]), and reduce communication overhead between PEs, these PEs are placed across memory channels for independent attention operands computations. Correspondingly, the memory-centric PIM design also leads to specific requirements on the physical layout of operands in memory, resulting in a unique data layout scheme, as shown

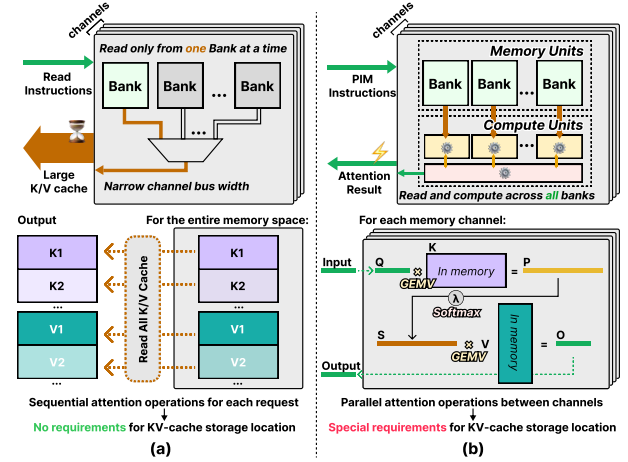


Fig. 1. Architectural changes cause data layout transformations. (a) Traditional memory architecture, (b) Processing-In-Memory architecture, and the respective data flows during attention operations.

in Fig.1. For instance, in the NeuPIMs [4] design, the entire attention computation for a single request must be completed within a single memory channel. Consequently, all KV cache associated with that request must be stored exclusively within the same channel, deviating from the conventional channel-interleaved memory storage approach used in standard memory architectures. While this design enhances computational efficiency within each channel, it introduces constraints on data placement that may affect overall system performance and flexibility.

We observed that in long-context inference scenarios, the existing **data layout scheme** presents two major issues. First, there exists memory fragmentation: each memory channel wastes space that is insufficient to store a complete KV cache, thereby reducing the number of requests that can be processed in parallel. Compared to the unified memory layout used in traditional GPUs, the current scheme leads to approximately 20% memory waste, resulting in an estimated 10% reduction in inference performance. The second issue is redundant computation, as channel-independent memory management prevents KV cache sharing across channels, undermining the effectiveness of the prefix caching mechanism. Furthermore, as context lengths continue to increase, the current layout inherently limits support for longer contexts. For example,

*Corresponding Author: Xueqi Li (lixueqi@ict.ac.cn). This work was supported in part by the National Natural Science Foundation of China under Grant No. 62488101 and 62202454, the National Key R&D Program of China under Grant 2022YFB4500403.

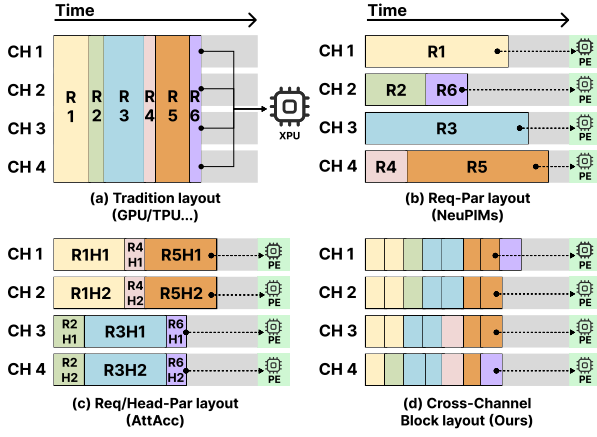


Fig. 2. Different memory layouts for distributing the KV cache of six request across four channels.

when running the Llama-3.1 70B model on a NeuPIM system with the existing layout, the maximum supported context length is only 25.6K tokens, significantly lower than the model’s potential of 128K tokens.

Driven by the **goal** of optimizing memory management to overcome these challenges, we proposed BlockPIM, a cross-channel block memory layout scheme, that maximizes memory utilization and eliminates the context length constraint. Moreover, BlockPIM does not incur any redundant storage and computation when applying prefix caching. To support distributed attention operation on BlockPIM, we proposed a cross-channel attention computation mechanism that only requires minor modifications to the existing architecture. Experimental results demonstrate that our approach achieve 62% throughput increase on average compared with existing PIM solution, enabling efficient and scalable deployment of large language models on PIM architectures.

II. BACKGROUND

A. Existing Memory Layout

During inference, multiple requests are typically grouped into a batch and processed in parallel. Each request stores intermediate values, known as the KV cache, in memory. Traditional XPU and PIM-solution adopt different strategies for organizing the KV caches in memory due to their differing approaches to computing attention.

1) *Traditional Layout*: In traditional memory, the KV cache of a request is evenly distributed across channels due to channel-level memory interleaving, as shown in Fig. 2(a). When performing attention on this request, XPU simultaneously reads different parts of the KV cache from different channels. Since each request has a unique KV cache, traditional XPU execute attention operations sequentially for each request when processing the attention layer. From a memory perspective, the XPU accesses the KV cache one at a time.

2) *Req-Par Layout*: In the NeuPIMs design, the KV cache for each request is stored within a single channel, while KV caches for different requests are distributed across channels.

A greedy algorithm assigns a new request’s KV cache to the channel with the lowest usage. For instance, as illustrated in Fig.2, assuming requests 1 through 6 are allocated sequentially, request 6 would be placed in the second channel, as it has the lowest usage after assigning the first five requests. During attention operations, channels perform computations in parallel, processing requests sequentially within each channel. We term this strategy **Req-Par**, as it exploits the independence of attention computations across requests, enabling parallel attention execution across channels.

3) *Req/Head-Par Layout*: The AttAcc memory layout strategy extends Req-Par layout by introducing parallelism across attention heads, as shown in Fig. 2(c). We use N_{kvhead}^{device} to represents the number of KV-heads allocated on this device under a tensor parallelism scheme. For each request, KV cache are stored across N_{kvhead}^{device} channels. When a new request is added to the batch, the KV cache are divided into N_{kvhead}^{device} segments along the attention head dimension and assigned to the lowest usage channels using a greedy algorithm. We term this strategy **Req/Head-Par** for exploiting independent computations across requests and attention heads. However, with the increasing adoption of *Grouped Query Attention*, the number of KV-heads has substantially decreased. Even the largest open-source model, Llama 3.1 405B, uses only 8 KV-heads, as shown in Table I. In an 8-way tensor parallelism setup, each device handles a single KV-head, reducing the Req/Head-Par layout effectively to a Req-Par layout. Therefore, for simplicity in discussion, all PIM layouts mentioned later are considered equivalent to the Req-Par layout.

B. Prefix Caching

Prefix Caching (also known as Prompt Caching) is a critical technique widely employed in modern inference frameworks such as vLLM [5], SGLang [6], TensorRT-LLM [7], LMDe-ploy [8]. By optimizing memory usage and reducing redundant computation, it effectively lowers response latency and enhances system throughput.

Inference systems often handle multiple requests that share the same prefix, leading to repeated computations for these prefixes. Additionally, storing multiple copies of identical prefix KV caches results in memory inefficiency. Based on this observation, Prefix Caching identifies and caches the attention computation results for these shared prefixes, enabling direct reuse in subsequent requests. This approach offers two main advantages: First, reusing cached KV caches reduces the computational load in the prefill stage, thereby decreasing response latency. Second, avoiding redundant storage of KV caches in memory allows the limited memory to accommodate more requests, increasing the batch size and further boosting system throughput.

III. MOTIVATION

A. Memory Underutilization

In the existing PIM memory layout, each request must be allocated to one or more fixed channels. If a new request is unable to fit into any channel, it can only wait for other

requests to release memory, even if the total available memory across channels is far greater than the space required by this request. This caused memory fragmentation for each channel, thus reducing the number of requests that can be processed in parallel and ultimately degrading inference performance.

Assuming a configuration of 40 channels, each with a capacity of fitting 32K tokens' KV cache, we conduct a simulation excluding any memory overheads beyond the KV cache. For this simulation, we select requests from the Mooncake [9] trace dataset where the total length does not exceed 32K tokens. According to the simulation results, the traditional layout achieves a memory utilization rate of 99.55%, while the Req/Head-par layout reduces this rate to 79.57%, with the average batch size decreasing from 141.37 to 112.88. When estimating based on a GPU platform, such a reduction in batch size implies an 18.8% decrease in throughput.

B. Inefficiency with Prefix Caching Algorithm

Prefix Caching builds upon a paged memory strategy, dividing memory into fixed-size pages that store KV caches. Pages may be shared across requests or dedicated to individual ones. Once all associated requests are completed, a page is marked releasable but retains data to improve reuse. When memory is full, the scheduler overwrites the least-reusable releasable page for new KV caches.

The Req-Par and Req/Head-Par layouts essentially forgo traditional channel-level memory interleaving, instead establishing an independent memory space for each channel. This approach results in incompatibility with Prefix Caching, which is designed for a unified memory space.

A direct adaptation is to implement paging independently within each channel, limiting page reuse to within channels. However, due to the greedy allocation scheme, requests with the same prefix may not be assigned to the same channel, preventing cross-channel page sharing. Consequently, the KV cache for shared prefixes is redundantly stored across channels, reducing the effectiveness of Prefix Caching and introducing repeated computation overhead. To make matters worse, due to the general applicability of large models, shared prefixes in inference services are often more than one. Multiple prefixes within a limited channel space compete with each other, and when memory is insufficient, the KV cache of existing prefixes is overwritten by the new request's prefix. This greatly reduces the likelihood of prefix reuse.

C. Limited Context Length

In the Req-Par layout, all KV cache for a single request is stored in a single memory channel. However, the capacity of a single memory channel is quite limited. As recent model context length increases rapidly, the memory usage of the KV cache for a single request can easily exceed the capacity of one memory channel. For a request with a length of L tokens, the memory usage of its KV cache on a device in FP16 precision is: $\text{Mem}_{\text{request}} = 2 \times N_{\text{layers}} \times D_{\text{head}} \times N_{\text{kvhead}}^{\text{device}} \times L \times 2$, where $N_{\text{kvhead}}^{\text{device}}$ represents the number of K/V heads allocated to this device after tensor parallelism. Given model parameters and

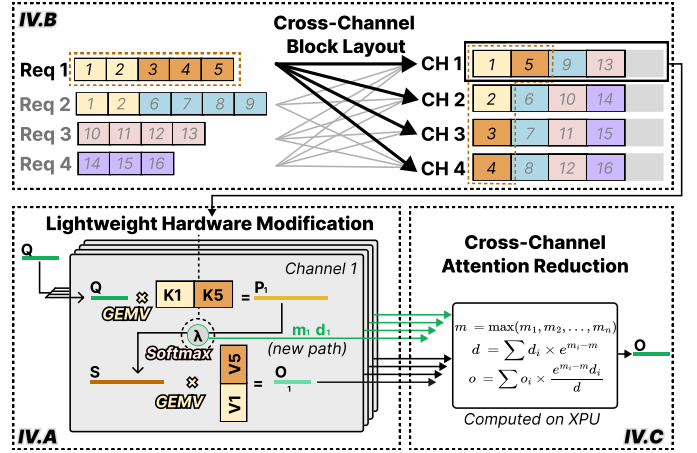


Fig. 3. BlockPIM design overview

channel capacity, we can estimate the maximum context length supported by a single channel in the Req-Par layout: $L_{\text{max}}^{\text{Req-Par}} = \text{Mem}_{\text{channel}} / (2 \times N_{\text{layers}} \times D_{\text{head}} \times N_{\text{kvhead}}^{\text{device}} \times 2)$.

In the Req/Head-Par layout, the KV cache corresponding to different KV heads of a request are distributed across different channels, reducing the memory pressure on a single channel. Thus, $L_{\text{max}}^{\text{Req/Head-Par}} = \text{Mem}_{\text{channel}} / (2 \times N_{\text{layers}} \times D_{\text{head}} \times 1 \times 2)$.

For a typical 8Gb HBM3 memory channel and various models, Table I below shows the maximum allowed context length under different Req-Par and Req/Head-Par layouts. It can be observed that, particularly for models supporting long contexts, the context length supported by these two memory layouts is far below the potential context length capability of the models. The context length of advanced large models has already surpassed 1M (e.g., Gemini Pro [10]), while the physical capacity of memory channels is challenging to increase proportionally, making it difficult for existing PIM architectures to support large models with long contexts under current memory layouts.

IV. BLOCKPIM DESIGN

As shown in Fig.3, BlockPIM consists of three key components: (1) Lightweight hardware modification, which gives architectural support for our cross-channel block layout scheme and cross-channel attention computation algorithm. (2) Cross-channel block layout, which reduces memory underutilization and redundancy in prefix caching, while overcoming context length limitations of traditional layouts with architectural and algorithmically support. (3) Cross-channel attention computation, ensuring correct attention results under the cross-channel block layout.

A. Lightweight Hardware Modification

We first briefly summarize the system architecture of the PIM-enabled computing system, then focusing on the lightweight architectural support introduced in our design.

System architecture. As illustrated in Fig.4(a)-(c), the PIM-device and XPU (e.g., GPU, TPU) are interconnected via

	Model Parameters				Contexts length	Maximum allowed contexts length				
	Name	D_{head}	$N_{kvheads}$	N_{layers}		Req-Par (No TP)	Req-Par (4-way TP)	Req-Par (8-way TP)	Req/Head-Par	BlockPIM
Short context Model	OPT 175B	128	96	96	2K	0.2K	0.9K	1.8K	21.3K	853.3K
	Gemma-2 27B	128	16	46	8K	2.8K	11.1K	22.3K	44.5K	1780.9K
Long context Model	Qwen-2 72B	128	8	80	128K	3.2K	12.8K	25.6K	25.6K	1024.0K
	Llama-3.1 70B	128	8	80	128K	3.2K	12.8K	25.6K	25.6K	1024.0K
	Mistral-Large-2 123B	128	8	88	128K	2.9K	11.6K	23.3K	23.3K	930.9K
	Llama-3.1 405B	128	8	126	128K	2.0K	8.1K	16.3K	16.3K	650.2K

TABLE I
CONTEXT LENGTH VARIATIONS FOR DIFFERENT MODELS UNDER VARIOUS MEMORY LAYOUT STRATEGIES

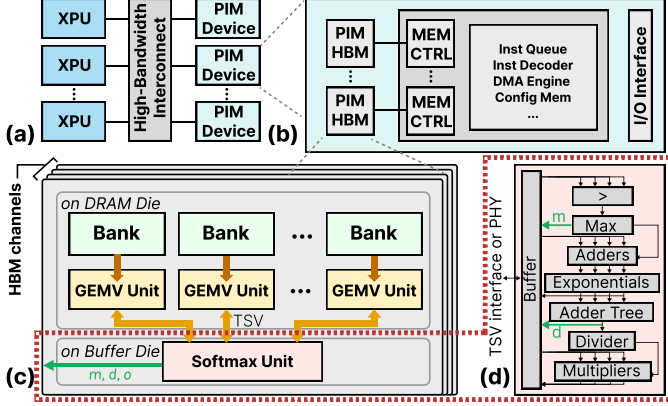


Fig. 4. Architecture of the PIM-enabled computing system in BlockPIM: (a) System hardware components, (b) PIM device structure, (c) Internal structure of PIM-enabled HBM channels, and (d) Softmax Unit with minor modifications (green arrow).

high-bandwidth links, such as NVLink or PCIe. Each PIM-device consists of multiple HBM memory chips, managed by a centralized control chip responsible for issuing memory access or processing-in-memory commands. Within each HBM channel, dedicated computing units for operations like GEMV and Softmax are embedded within the DRAM die or buffer die to collaboratively perform attention computation.

Lightweight modification. In Fig.4(d), our modification targets the internal structure of the Softmax unit. When the maximum value m of the input vector and the denominator d are computed, instead of simply passing these values for subsequent calculations, we introduce additional connections (green arrow in the figure) to the buffer and modify the control logic, enabling m and d to be directly stored in the buffer. After completing the Softmax computation, m and d , along with the output vector o , are transferred to the XPU to continue cross-channel attention computation, as detailed in IV-C.

B. Cross-channel Block Memory Layout

Layout Strategy. We propose the cross-channel block layout to address limitations in existing memory layouts. In this design: (1) The KV cache generated for each new request is divided into fixed-size blocks, with each block representing a fixed number of tokens. (2) These blocks are distributed

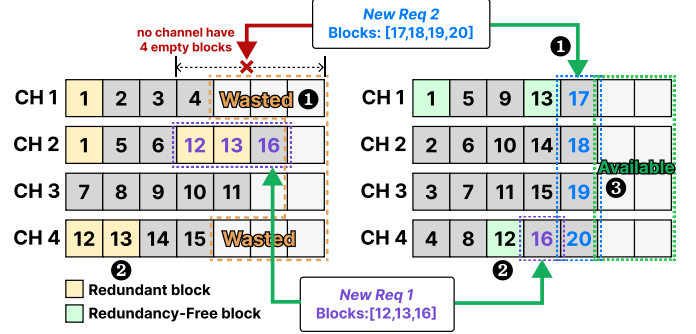


Fig. 5. Comparison of Req-Par block layout from NeuPIMs (left) and cross-channel block layout (right) highlighting the advantages of the latter.

sequentially across memory channels using a greedy strategy, which allocates each block to the channel with the least current capacity utilization. (3) During inference, newly generated KV cache blocks are similarly assigned using the greedy strategy, ensuring that the KV caches of all requests remain approximately balanced across channels.

In contrast to NeuPIMs [4], which employs a block management strategy akin to PagedAttention [5], the cross-channel block layout optimizes block distribution across channels rather than confining them within individual channels. NeuPIMs' isolated block management perpetuates inefficiencies found in Req-Par and Req/Head-Par layouts. By balancing KV cache blocks across channels, the cross-channel block layout effectively addresses these inefficiencies, offering a more coordinated and efficient memory strategy.

Advantages. Our proposed layout implements a unified, cross-channel block management system, which brings three significant advantage, as shown in Fig. 5: ① The placement of new request's KV cache is no longer limited by the remaining space of a single channel, which improves the utilization of memory space. This avoids wasting space in each memory channel that cannot accommodate an entire KV cache. In other words, under our layout, these wasted spaces are used reasonably due to the finer-grained allocation method. ② No redundant computation or storage in Prefix Caching. Under BlockPIM's layout, a new request can directly reuse the KV cache of a shared prefix stored in any location, rather than

being limited to the space within a single channel. This means that the shared prefix only needs to be computed and stored once, eliminating redundant storage across channels. Moreover, the shared prefix KV cache no longer needs to compete for space within a single memory channel, significantly reducing KV cache invalidation and improving reuse rates. ③ A request's KV cache can be distributed across multiple channels, maximizing the utilization of all available storage, akin to traditional layouts, which effectively mitigates the context length limitation in existing layouts. With this cross-channel layout, the capacity limit for a single request's KV cache extends beyond the constraint of a single channel, equating to the cumulative size of all available memory blocks within the system.

Block Size Considerations. To determine block size, it is essential to consider DRAM memory access characteristics, particularly the impact of row switching. Since accessing data from different rows incurs significant latency, it is beneficial to utilize all data within a single row before switching to another. Thus, consecutive K or V cache data is stored in a single row of each DRAM bank. However, we store the K and V cache in separate rows of each DRAM bank, as the Softmax computation between reading the K and V cache helps hide the row switching latency. As a result, the block size is twice the product of the total number of banks of a channel and the capacity of a single bank row.

C. Cross-channel Attention Computation

Computation Steps. For the attention computation process of a request, our cross-channel attention computation is divided into three steps: ① Within each channel, all KV cache blocks belonging to the request (denoted as K_i and V_i , where i represents the channel index) compute attention with the input Q vector, yielding the local attention result $O_i = \text{Softmax}\left(\frac{QK_i^T}{\sqrt{d_k}}\right)V_i$. Each channel's buffer simultaneously records two intermediate value m_i and d_i during the attention process. Here, m_i is the maximum value used in softmax, and d_i is the softmax normalization base value. After computation, O_i , m_i , and d_i are transmitted to the XPU. ② On the XPU, the global m and d values are computed sequentially using the following formulas: $m = \max(m_1, m_2, \dots, m_n)$, $d = \sum d_i \times e^{m_i - m}$. ③ On the XPU, based on the global m and d , the partial results from each channel are corrected and reduced to obtain the complete attention computation result: $O = \sum O_i \times \frac{e^{m_i - m} d_i}{d}$.

Our cross-channel attention computation enables the attention operation of a request to be decomposed and computed in parallel across multiple channels. As a result, this approach eliminates the need for the operands of the attention operation, the complete KV cache, to be stored entirely within a single channel.

Algorithmically correctness. Our approach ensures correct attention computation under a cross-channel block layout. The conventional attention operation, $\text{Attention}(Q, K, V)$, is generally divided into three main steps: $S = \frac{QK^T}{\sqrt{d_k}}$, $P =$

$\text{Softmax}(S)$, $O = PV$. Under the cross-channel block layout, the K and V matrices are segmented along the sequence length dimension. For a given channel i , the partial output is computed as O_i . However, the overall output O does not simply equal $\sum O_i$; the partial results from each channel cannot be directly summed. This discrepancy arises because the original attention computation requires a softmax operation over the entire global S vector, while each channel i can only utilize the local S_i data. Thus, the attention outputs from each channel must undergo correction prior to summation. The step (2) of our algorithm calculates the correction coefficients, and the step (3) performs the correction and summation, thus ensuring the correctness of the final results.

Extra Overhead. In addition to transmitting the partial result O_i to the host, each softmax unit must also convey the parameters d_i and m_i . Fortunately, transmitting two additional values per channel has minimal impact on communication overhead. The computational cost of the extra operation is also minimal, capped by the correction and summation of d_{head} -length vectors across all channels, regardless of the request length. We extended the supported operators of LLMCompass [11] to simulate this overhead in experiments.

V. EVALUATION

A. Experimental Setup

Simulation Settings. (1) Software simulation: An inference engine simulator was implemented which faithfully incorporated different memory layout strategies. The simulation employed a First-Come-First-Serve (FCFS) policy to handle incoming inference requests. The evaluation is conducted on the representative long-context model, Llama-3.1 70B. (2) Hardware simulation: The AttAcc simulator [3], built on the cycle-accurate memory simulator Ramulator [12], was utilized to model processing-in-memory (PIM) behavior. For the XPU component, we leveraged LLMCompass [11], a specialized simulator designed for evaluating hardware performance in LLM inference tasks. LLMCompass is calibrated against real hardware performance metrics to ensure accuracy.

Block Size Configuration. In our configuration, each HBM channel has 64 banks, and each bank row stores 1KB of data. This means the read/compute unit for each all-bank PIM operation is 64KB. With FP16 precision, each token requires 128x2B of memory for the K or V cache, allowing 64KB to store data for 256 tokens. Thus, the total block size, which stores both the K and V cache for 256 tokens, is 128KB.

Datasets. (1) ShareGPT [13]: Publicly shared ChatGPT logs, featuring short-input, short-output interactions. Since ChatGPT's system prompt is private, we use Claude 3.5 Sonnet's official system prompt [14] (2,658 tokens) as an approximate prefix for user requests. (2) Mooncake [9]: Real-world inference traces from Moonshot, with medium-to-long inputs and short outputs. The dataset itself includes details on prefix reuse between requests. (3) LongWriter [15]: A dataset for long-text generation, emphasizing short-input, long-output scenarios. There is no prefix reuse in this dataset. (4) LooGLE [16]: A benchmark dataset for long-context

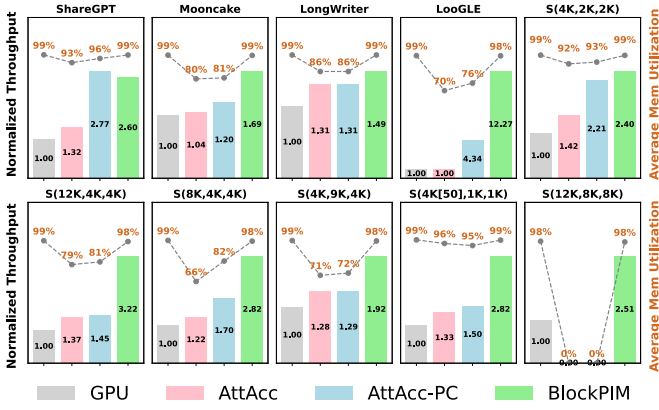


Fig. 6. Performance Comparison

models, highlighting long-input, short-output cases. We only use the Q/A test samples for long context understanding, where each long text is reused for different Q/A pairs. (5) Synthetic datasets: Given the scarcity of public long-context inference trace datasets, we generated six synthetic datasets $S(prefix, input_{mean}, output_{mean})$ for more comprehensive comparisons across different data characteristics. Each synthetic dataset has 1,000 samples with input and output lengths normally distributed. In $S(4K[50], 1K, 1K)$, “50” denotes 50 distinct system prompts.

Baselines. (1) GPU: A standard DGX server configuration with $8 \times A100$ GPUs. (2) AttAcc: Extends the GPU configuration with 8 PIM devices, each equipped with 40GB of HBM3 memory (1GB per channel) and memory timing aligned with AttAcc specifications. (3) AttAcc-PC: In addition to the AttAcc configuration, prefix caching are enabled per channel.

B. Performance Comparison

As shown in Fig. 6, we compare the acceleration achieved by our approach relative to the baseline across different datasets. Each set of data can also serve as an ablation study, reflecting the individual performance gains contributed by PIM, prefix caching, and our optimization. In the ShareGPT dataset, with its short system prompts, inputs, and outputs, memory fragmentation and redundant computation issues are minimal, resulting in comparable speedups for BlockPIM and AttAcc-PC, with BlockPIM being slightly lower ($2.77x \rightarrow 2.60x$) due to the extra overhead. However, across all other long-context datasets, BlockPIM consistently delivers performance improvements. In the LongWriter dataset, which does not use shared prefixes, the performance improvement of BlockPIM over AttAcc-PC ($1.31x \rightarrow 1.48x$) is entirely attributed to more efficient utilization of memory resources. In the LooGLE dataset, which highlights extreme cases of shared ultra-long prefixes, BlockPIM achieves a significant performance gain by avoiding redundant computation/storage, with the speedup increasing from 4.33x to 12.2x compared

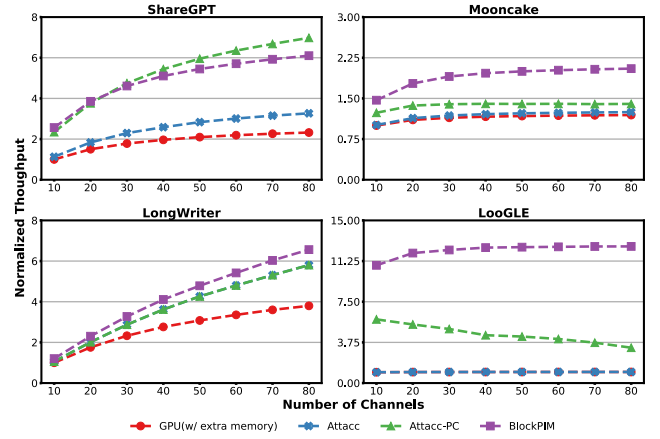


Fig. 7. Scalability Analysis

to AttAcc-PC. For the Mooncake dataset, the performance improvement ($1.20x \rightarrow 1.68x$) results from the combined effect of addressing both memory fragmentation and redundant computation issues. Our first five synthetic datasets demonstrate the acceleration of BlockPIM over AttAcc-PC under long input and output scenarios, with the average speedup improving from 1.62x to 2.63x. Due to AttAcc’s limitations in context-length, it cannot process the final synthetic dataset, whereas our BlockPIM effectively achieves a 2.51x speedup.

C. Scalability Analysis

As shown in Fig.7, we evaluated the normalized throughput of AttAcc, AttAcc-PC, and BlockPIM as PIM memory channels increased, with the GPU (with extra memory) configuration serving as a baseline to reflect the performance improvement attributed solely to increased memory capacity. (1) ShareGPT dataset: AttAcc-PC and BlockPIM achieved similar scalability, showing that BlockPIM’s optimization for long-context inference does not hinder short-context performance. (2) Mooncake dataset: Performance gains from additional PIM channels were limited due to XPU bottlenecks in long-input, short-output tasks. However, BlockPIM still demonstrated better scalability than AttAcc-PC. (3) LongWriter dataset: BlockPIM consistently outperformed AttAcc-PC by leveraging memory resources more efficiently. (4) LooGLE dataset: AttAcc-PC performance decreased with more channels, as prefix reuse efficiency dropped due to distribution across channels, leading to redundant computations. BlockPIM avoided this issue, maintaining stable performance.

VI. CONCLUSION

This paper proposed BlockPIM, a novel memory management scheme for PIM-enabled LLM inference system. BlockPIM maximizes memory utilization, eliminates redundant storage and computation in prefix caching and removes context length constraints. Results show that BlockPIM significantly outperforms the state-of-the-art PIM-solution in long-context scenarios.

REFERENCES

- [1] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. Vijaykumar, "Newton: A dram-maker's accelerator-in-memory (aim) architecture for machine learning," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 372–385.
- [2] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin *et al.*, "Hardware architecture and software stack for pim based on commercial dram technology: Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 43–56.
- [3] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "Attacc! unleashing the power of pim for batched transformer-based generative model inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 103–119.
- [4] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 722–737.
- [5] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626.
- [6] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez *et al.*, "Sglang: Efficient execution of structured language model programs," *arXiv preprint arXiv:2312.07104*, 2023.
- [7] NVIDIA, "TensorRT-LLM," <https://github.com/NVIDIA/TensorRT-LLM>, 2024, version 0.14.0, Accessed: Nov. 17, 2024.
- [8] InternLM, "LMDeploy," <https://github.com/InternLM/lmdeploy>, 2024, version 0.6.3, Accessed: Nov. 17, 2024.
- [9] R. Qin, Z. Li, W. He, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: Kimi's kvcache-centric architecture for llm serving," *arXiv e-prints*, pp. arXiv–2407, 2024.
- [10] G. Team, P. Georgiev, V. I. Lei, R. Burnell, L. Bai, A. Gulati, G. Tanzer, D. Vincent, Z. Pan, S. Wang *et al.*, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," *arXiv preprint arXiv:2403.05530*, 2024.
- [11] H. Zhang, A. Ning, R. B. Prabhakar, and D. Wentzlaff, "Llmcompass: Enabling efficient hardware design for large language model inference," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 1080–1096.
- [12] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," *IEEE Computer architecture letters*, vol. 15, no. 1, pp. 45–49, 2015.
- [13] S. Team, "ShareGPT," <https://sharegpt.com/>, 2023.
- [14] Anthropic, "System prompts," <https://docs.anthropic.com/en/release-notes/system-prompts>, claude 3.5 Sonnet, Accessed: Oct. 22, 2024.
- [15] Y. Bai, J. Zhang, X. Lv, L. Zheng, S. Zhu, L. Hou, Y. Dong, J. Tang, and J. Li, "Longwriter: Unleashing 10,000+ word generation from long context llms," *URL https://arxiv.org/abs/2408.07055*, 2024.
- [16] J. Li, M. Wang, Z. Zheng, and M. Zhang, "Loogle: Can long-context language models understand long contexts?" *arXiv preprint arXiv:2311.04939*, 2023.